UNIVERSITY OF CEBU (MAIN)

SOFTWARE TESTING | SEMI-FINAL REQUIREMENT

TEST REPORT

User Login System: Register, Login and Logout

Web application built with Python Flask, SQLite and W3.CSS

Prepared by	Mandawe, Ivan Jae D.
School	University of Cebu (Main)
Subject / Activity	Software Testing, Semi-Final
Date of report	November 9, 2026
Application tested	UserLogin v1.0 (first build), then v1.1 (after defect fixes)
Test types covered	Smoke, Functional, Non-functional, Sanity, Regression, Confirmation (Re-testing) and Exploratory testing
Overall outcome	v1.0: 30 of 37 test cases passed, 7 defects found. v1.1: 37 of 37 test cases passed.

1. Introduction

This report documents the testing of a simple web application that lets a person register an account, log in and log out. The activity requires the application to be tested with functional, non-functional, smoke, sanity, regression, confirmation (re-testing) and exploratory testing, with the procedures, expected results, actual results and findings written down together with screenshots.

1.1 The application under test

The application follows the structure shown in the class tutorial video (Flask + SQLite + W3.CSS) and adds the registration page that the tutorial does not include. Its pages are:

- Login page (/): e-mail and password form, with a link to the register page.
- Register page (/register): full name, e-mail, password and confirm password.
- Main page (/main): protected page that shows the students table and a LOG-OUT button. It is reachable only while logged in.
- Logout (/logout): clears the session and returns to the login page.

The footer of every page reads: Copyright © Mandawe, Ivan Jae D. 2026.

1.2 Test environment

Item	Details
Application	UserLogin (Python 3.12.3, Flask 3.1.3, Werkzeug 3.1.9, SQLite database school.db)
Browser used for testing	Chromium 153 driven automatically by Playwright 1.63 (desktop viewport 900 x 520; mobile test at 375 x 667)
Operating system	Ubuntu 24.04 LTS
Server	Flask development server on http://127.0.0.1:5057 (localhost)
Test data	Fresh database for every test run. Seed account alpha@uc.com / hello123; six sample students (IDNO 1000 to 1005).
How tests were run	Each test case was executed by a script (tests/run_tests.py) that performs the steps in a real browser, records the actual result and saves a screenshot.

2. Test approach

The testing was done in two rounds. Round 1 tested the first build (v1.0) with every test case. The defects found were fixed in v1.1, and Round 2 then checked the fixes with sanity, confirmation and regression testing.

Test type	What it checks	Where it was applied
Smoke testing	A quick check that the application starts and its most important functions work, before any deeper testing.	SM-01 to SM-05, run first in Round 1.
Functional testing	Each feature (register, login, logout, protected page) behaves as specified, including invalid input.	FT-01 to FT-12 in Round 1 and in the regression run.
Non-functional testing	Quality attributes: performance, security, usability on mobile, reliability.	NF-01 to NF-08 in Round 1 and in the regression run.
Exploratory testing	Unscripted, curiosity-driven testing with unusual inputs and navigation to find problems that scripted cases miss.	EX-01 to EX-12. This is where all 7 failing cases were found.
Sanity testing	A short, narrow check after a fix to confirm the changed build is reasonable before doing a full retest.	6 cases on v1.1 right after the fixes (section 6).
Confirmation (re-testing)	Re-running the exact test cases that failed, to confirm the defects are really fixed.	EX-01 to EX-07 re-run on v1.1 (section 7).
Regression testing	Re-running the whole suite on the new build to make sure the fixes did not break anything that worked before.	All 37 cases re-run on v1.1 (section 8).

Test cycle: Round 1 on v1.0 (smoke, functional, non-functional, exploratory) → defect log → fixes in v1.1 → sanity → confirmation → regression.

3. Summary of results

Test type	Cases	v1.0 passed	v1.0 failed	v1.1 passed	v1.1 failed
Smoke	5	5	0	5	0
Functional	12	12	0	12	0
Non-functional	8	8	0	8	0
Exploratory	12	5	7	12	0
TOTAL	**37**	**30**	**7**	**37**	**0**

Findings in one paragraph. The main features (register, login, logout, access control) and the security and performance checks passed on the first build. Exploratory testing found 7 failing cases, which trace back to 6 defects, all in how the registration and login forms handle unusual input and in what a logged-in user sees. All six were fixed in v1.1; the 7 failed cases now pass and the full suite passes with no regressions.

4. Round 1: test cases and results (v1.0)

Every test case below was executed on the first build (v1.0). Each card shows the procedure, the expected result, the actual result that was observed and a screenshot taken during the test.

4.1 Smoke testing

Run first. If these fail there is no point continuing with deeper tests.

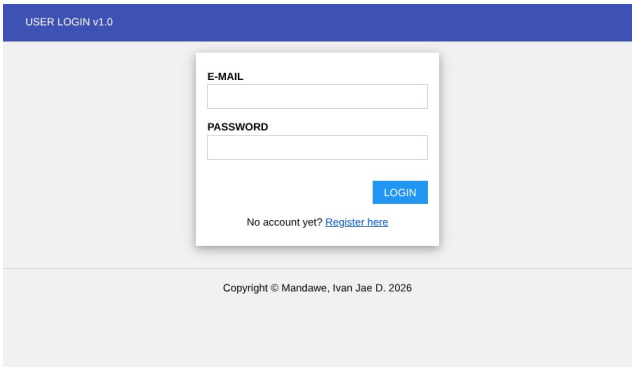
SM-01 Application starts and home page loads

RESULT: PASS

Procedure
Open http://127.0.0.1:5057/ in the browser.

Expected result
HTTP 200, page title 'USER LOGIN', login form (e-mail, password, LOGIN) visible.

Actual result
HTTP 200; title 'USER LOGIN'; header shows 'USER LOGIN v1.0'; login form visible: True.



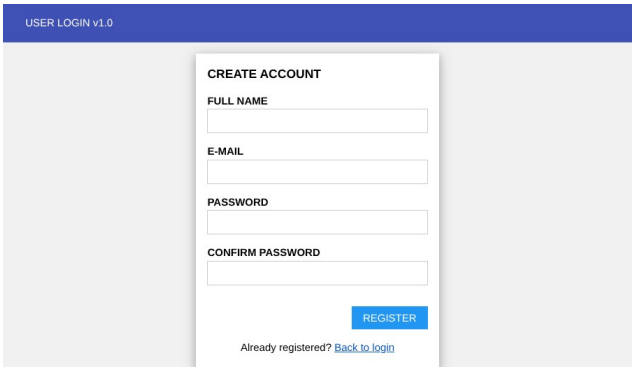
SM-02 Register page is reachable from the login page

RESULT: PASS

Procedure
On the login page click the 'Register here' link.

Expected result
Browser goes to /register and the CREATE ACCOUNT form is displayed.

Actual result
URL is http://127.0.0.1:5057/register; register form visible: True.



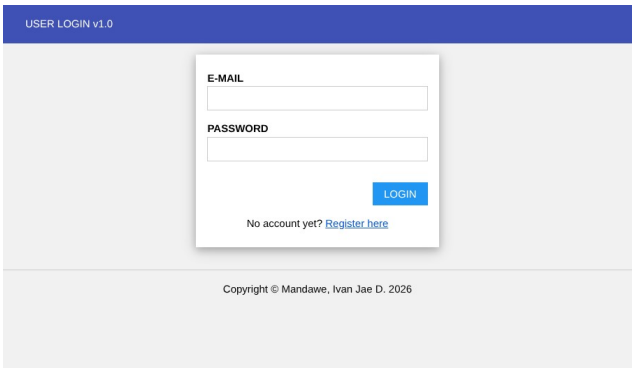
SM-03 Stylesheet loads

RESULT: PASS

Procedure
Request /static/css/w3.css.

Expected result
HTTP 200 and CSS content returned.

Actual result
HTTP 200; stylesheet contains the w3-* classes: True.

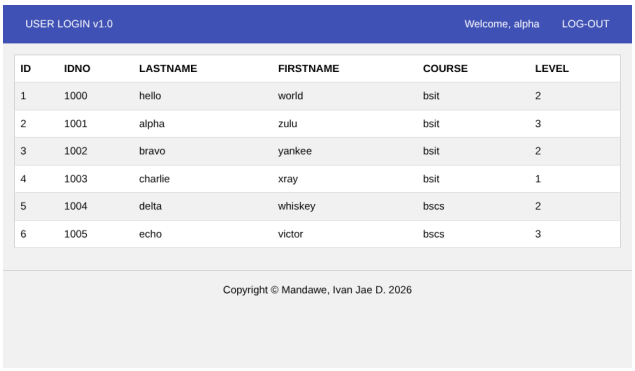


SM-04 Critical path: seeded user can log in	RESULT: PASS

Procedure
Enter alpha@uc.com / hello123 on the login page and press LOGIN.

Expected result
User is taken to /main and the students table is shown.

Actual result
Landed on http://127.0.0.1:5057/main; students table visible: True.

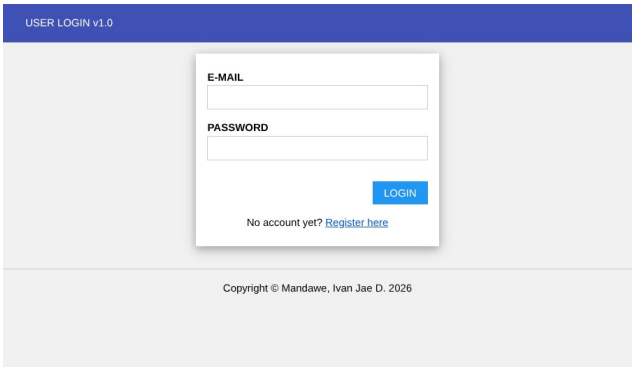


SM-05 Footer shows the correct copyright name	RESULT: PASS

Procedure
Open the login page and read the footer.

Expected result
Footer reads 'Copyright © Mandawe, Ivan Jae D. 2026'.

Actual result
Footer text: 'Copyright © Mandawe, Ivan Jae D. 2026'.



4.2 Functional testing

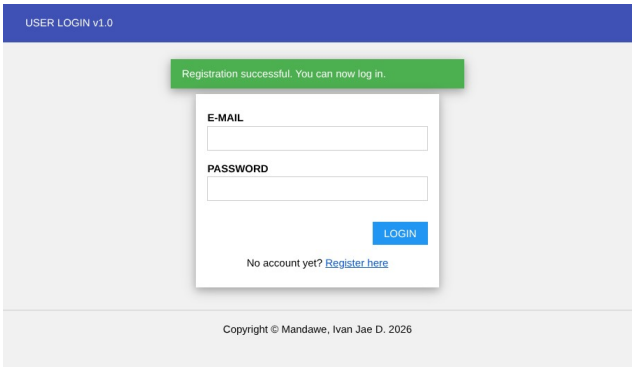
Checks each feature against its expected behaviour, including wrong and missing input.

FT-01 Register with valid details	RESULT: PASS

Procedure
Open /register; enter Name 'Juan Dela Cruz', E-mail juan@uc.com, Password Passw0rd! (twice); press REGISTER.

Expected result
Redirected to login page with a green 'Registration successful' message; one new row saved in the users table.

Actual result
Now on http://127.0.0.1:5057/; message: 'Registration successful. You can now log in.'; rows in users for juan@uc.com: 1.



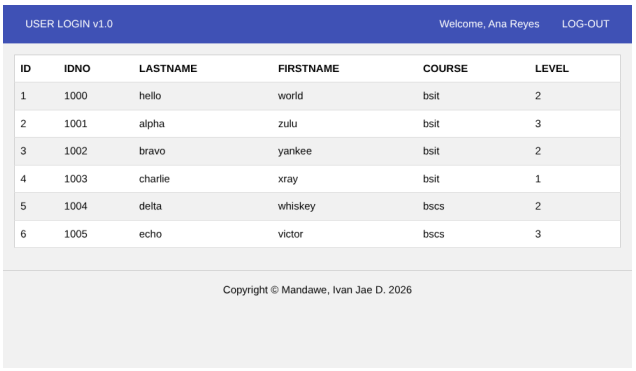
FT-02 Login with a newly registered account

RESULT: PASS

Procedure
Register ana@uc.com, then log in with ana@uc.com / Passw0rd!.

Expected result
Redirected to /main; header shows 'Welcome, Ana Reyes'.

Actual result
URL http://127.0.0.1:5057/main; header text: 'Welcome, Ana Reyes'.



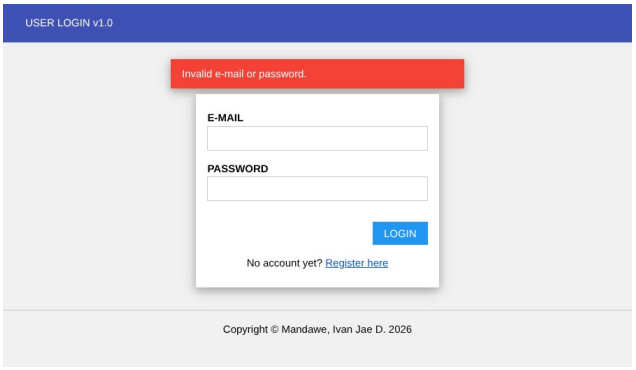
FT-03 Login with a wrong password

RESULT: PASS

Procedure
Enter alpha@uc.com with password 'wrongpass' and press LOGIN.

Expected result
Login refused; red 'Invalid e-mail or password.' message; /main stays inaccessible.

Actual result
Message: 'Invalid e-mail or password.'; /main redirected to login: True.



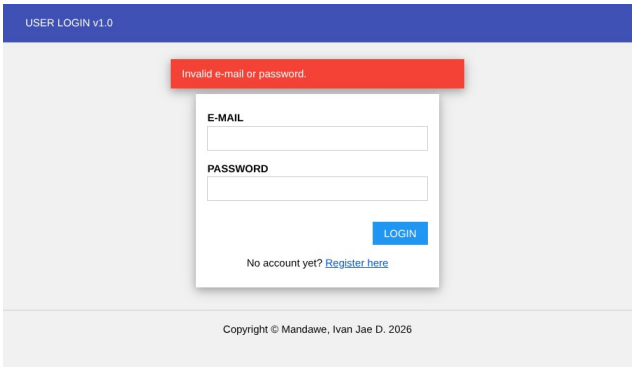
FT-04 Login with an unregistered e-mail

RESULT: PASS

Procedure
Enter nobody@uc.com / Passw0rd! and press LOGIN.

Expected result
Login refused with the same generic 'Invalid e-mail or password.' message.

Actual result
Message: 'Invalid e-mail or password.'; URL http://127.0.0.1:5057/.



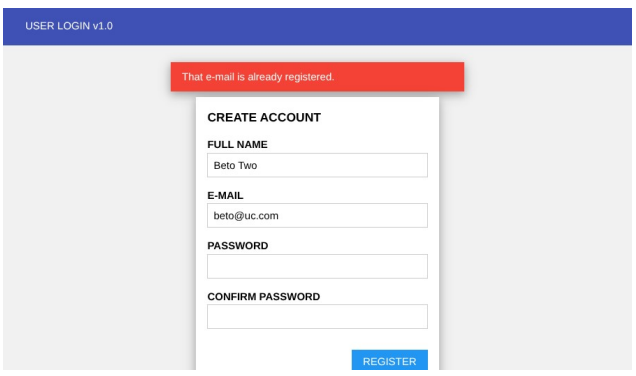
FT-05 Register with an e-mail that already exists

RESULT: PASS

Procedure
Register beto@uc.com once, then register beto@uc.com again with another name.

Expected result
Second attempt refused with 'That e-mail is already registered.'; only one row exists.

Actual result
Message: 'That e-mail is already registered.'; rows with that e-mail: 1.



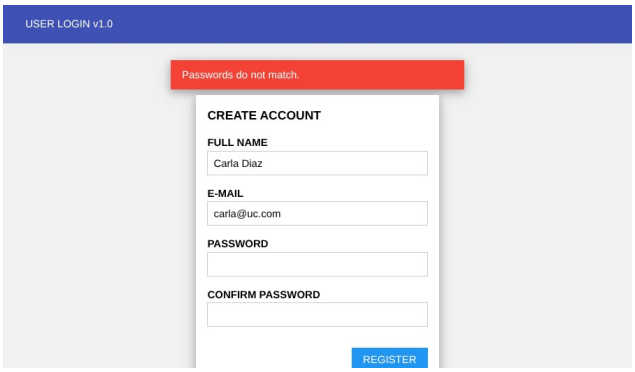
FT-06 Register with mismatched passwords

RESULT: PASS

Procedure
Register carla@uc.com with Password 'Passw0rd!' and Confirm 'Different1!'.

Expected result
Refused with 'Passwords do not match.'; no row saved; entered name/e-mail are kept in the form.

Actual result
Message: 'Passwords do not match.'; rows saved: 0; e-mail field kept: True.



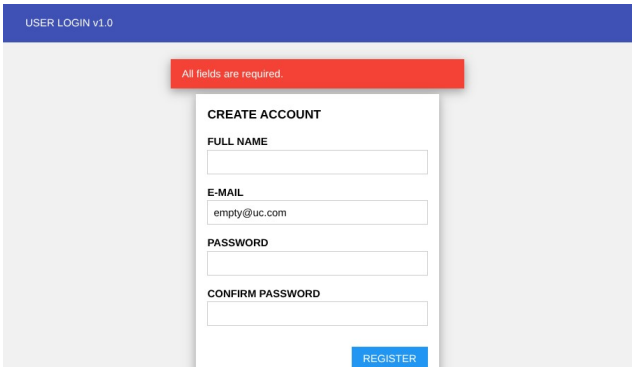
FT-07 Required field validation (server side)

RESULT: PASS

Procedure
Send a registration request with an empty Name field (bypassing the browser's 'required' check).

Expected result
Refused with 'All fields are required.'; no row saved.

Actual result
Response contains the required-fields message: True; rows saved: 0.



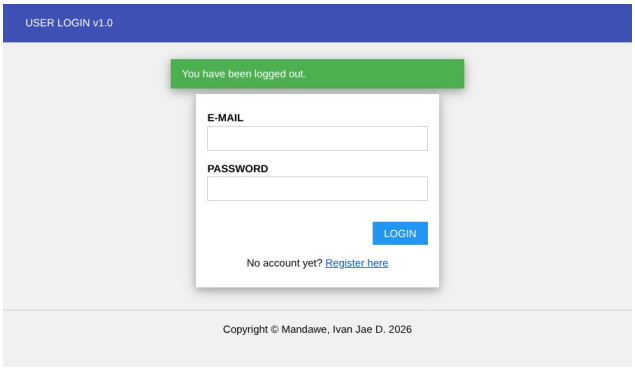
FT-08 Logout

RESULT: PASS

Procedure
Log in as alpha@uc.com, then click LOG-OUT.

Expected result
Redirected to the login page with 'You have been logged out.'; /main is no longer accessible.

Actual result
After logout URL http://127.0.0.1:5057/; message 'You have been logged out.'; /main now redirects to login: True.



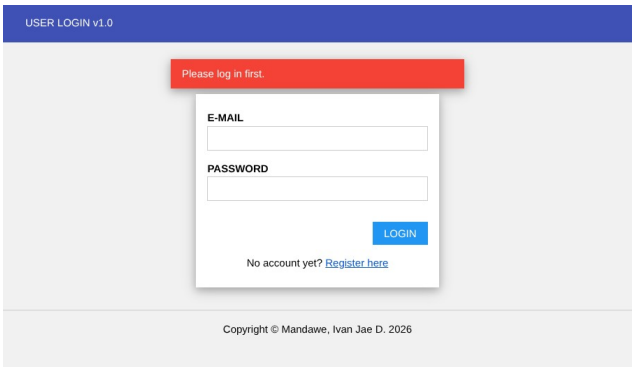
FT-09 Protected page without logging in

RESULT: PASS

Procedure
In a new browser session open /main directly.

Expected result
Redirected to login page with 'Please log in first.'

Actual result
URL http://127.0.0.1:5057/; message 'Please log in first.'.



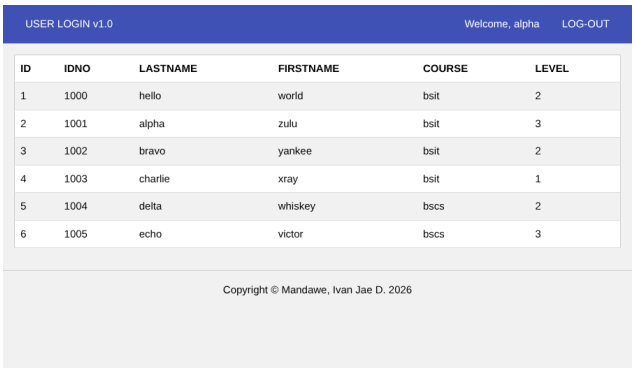
FT-10 Main page lists all students

RESULT: PASS

Procedure
Log in as alpha@uc.com and view /main.

Expected result
Table shows a header row plus 6 student rows (IDNO 1000-1005).

Actual result
Table rows counted (including header): 7.



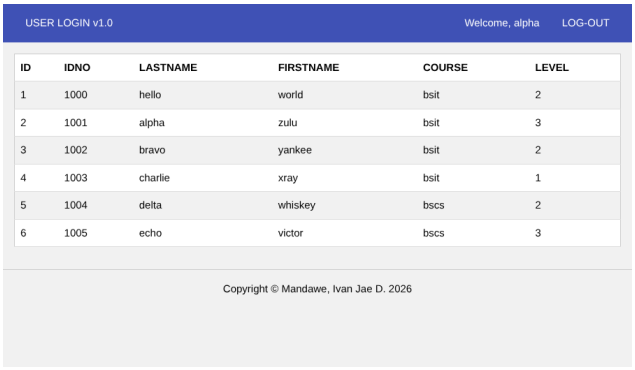
FT-11 Session persists after page refresh

RESULT: PASS

Procedure
Log in, then press the browser refresh button on /main.

Expected result
User remains logged in and the table is still shown.

Actual result
After reload URL http://127.0.0.1:5057/main; table visible: True.



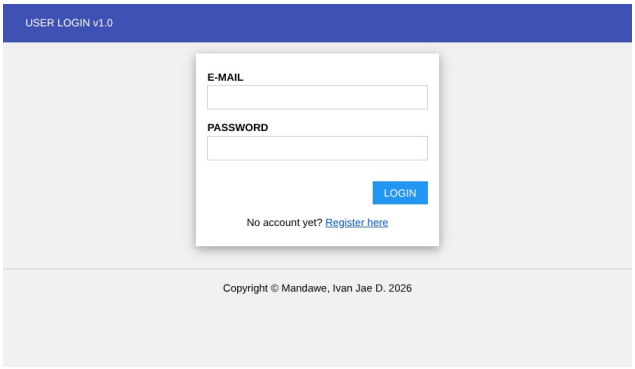
FT-12 'Back to login' link on the register page

RESULT: PASS

Procedure
Open /register and click 'Back to login'.

Expected result
Login page is displayed.

Actual result
URL http://127.0.0.1:5057/; login form visible: True.



4.3 Non-functional testing

Checks response time, password storage, injection attacks, cookies, mobile layout and reliability.

NF-01 Performance: page response time

RESULT: PASS

Procedure

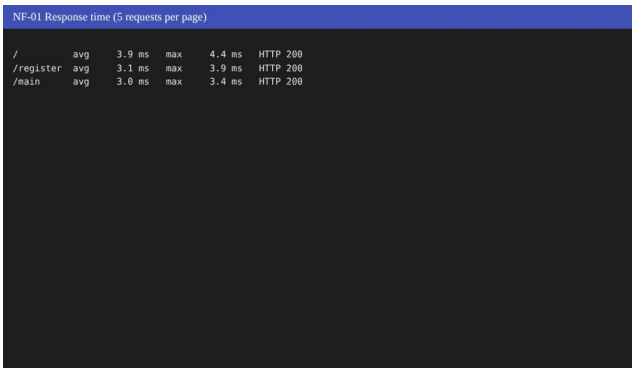
Request /, /register and /main five times each and measure the average response time.

Expected result

Average response time under 1000 ms for every page.

Actual result

/: avg 4 ms; /register: avg 3 ms; /main: avg 3 ms.



NF-02 Security: passwords are stored hashed

RESULT: PASS

Procedure

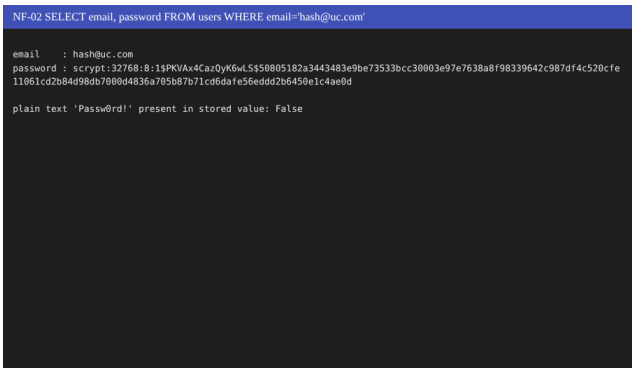
Register hash@uc.com with password 'Passw0rd!' and inspect the saved row in the database.

Expected result

The stored value is a salted hash, never the plain-text password.

Actual result

Stored value starts with 'scrypt:32768:8:1$PKV…' (162 characters); plain text not present.



NF-03 Security: SQL injection on login

RESULT: PASS

Procedure

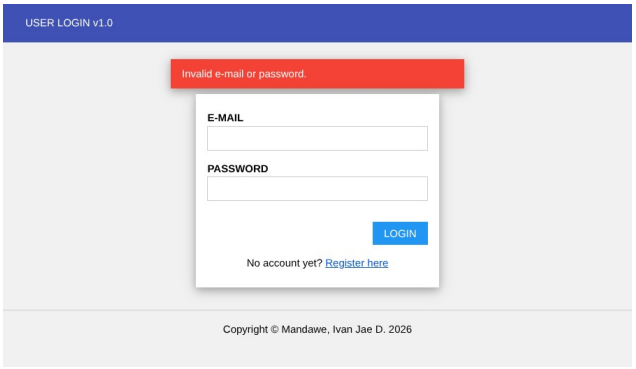
Send a login request with e-mail ' OR '1'='1 and any password.

Expected result

Login refused; no session created.

Actual result

Response contained 'Invalid e-mail or password.': True; /main reachable afterwards: False.



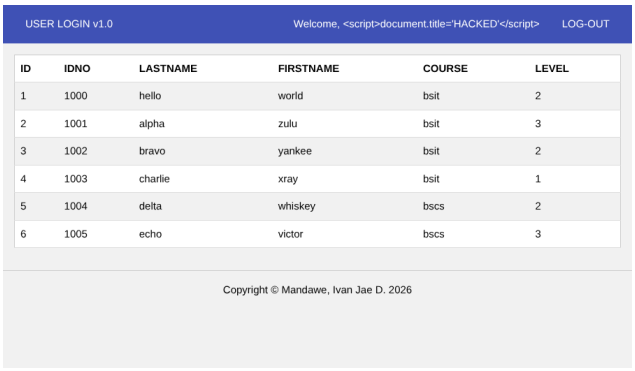
NF-04 Security: script injection (XSS) in the name field

RESULT: PASS

Procedure
Register with the name <script>document.title='HACKED'</script>, log in and view /main.

Expected result
The text is shown as plain text (escaped); no script runs; page title unchanged.

Actual result
Page title 'USER LOGIN'; header shows literal text: 'Welcome, <script>document.title='HACKED'</script>'.



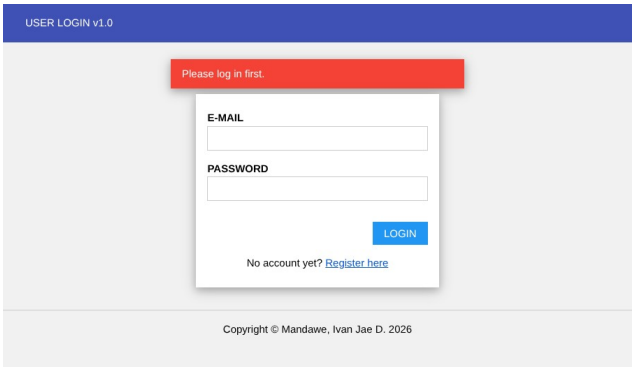
NF-05 Security: Back button after logout

RESULT: PASS

Procedure
Log in, open /main, click LOG-OUT, then press the browser Back button.

Expected result
Protected content is not shown again; user is sent back to the login page. /main is sent with no-store headers.

Actual result
Cache-Control on /main: 'no-cache, no-store, must-revalidate'; students table visible after Back: False; URL http://127.0.0.1:5057/.



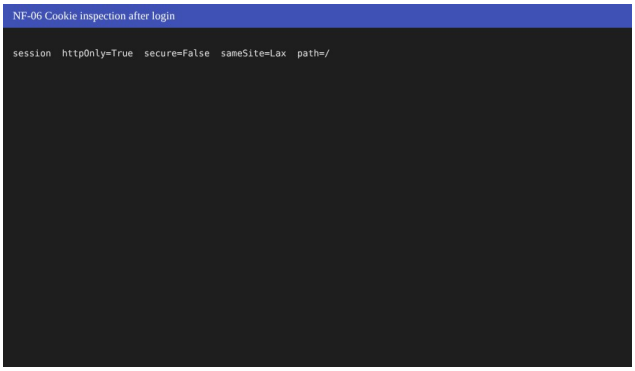
NF-06 Security: session cookie flags

RESULT: PASS

Procedure
Log in and inspect the 'session' cookie.

Expected result
Cookie exists and is flagged HttpOnly (not readable by JavaScript).

Actual result
Session cookie present: True; HttpOnly: True.



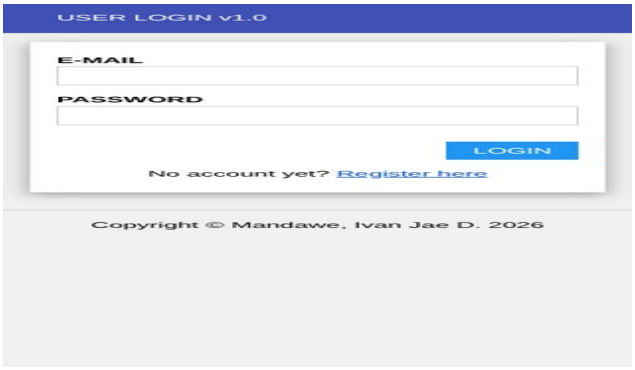
NF-07 Usability: mobile layout (375 px wide)

RESULT: PASS

Procedure
Resize the viewport to 375 x 667 and open the login and register pages.

Expected result
No horizontal scrolling; form fits inside the screen.

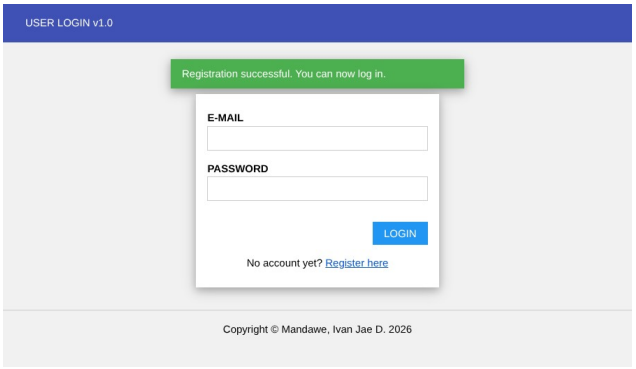
Actual result
/register: page width 375px, form right edge 359px; /: page width 375px, form right edge 359px (limit 375px).

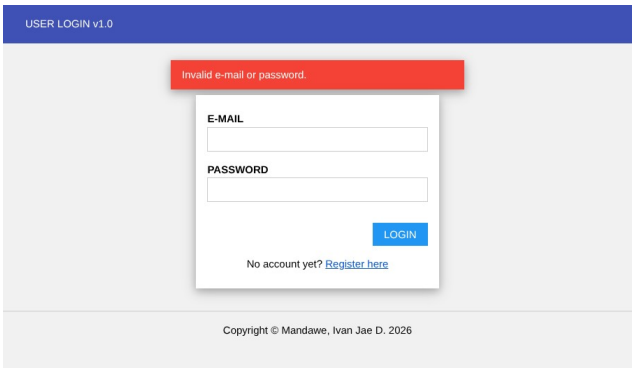


NF-08 Reliability: 25 consecutive logins	RESULT: PASS
Procedure Send 25 valid login requests one after another. **Expected result** All requests succeed with no server error (HTTP 5xx). **Actual result** 25 requests, statuses [200]; server errors: 0.	

4.4 Exploratory testing

Unscripted probing with unusual input. Each session had a short goal (a charter) and the findings were written down as test cases. Results marked FAIL are defects, listed in section 5.

EX-01 Same e-mail typed with different letter case	RESULT: FAIL
Procedure Register Case@Test.com, then try to register case@test.com. **Expected result** Second registration refused (e-mail addresses are not case-sensitive). **Actual result** Message: 'Registration successful. You can now log in.'; accounts now stored for this address: 2.	

EX-02 Log in using different letter case than at registration	RESULT: FAIL
Procedure Register Mixed@Case.com, then log in with mixed@case.com. **Expected result** Login succeeds. **Actual result** URL after login: http://127.0.0.1:5057/; message: 'Invalid e-mail or password.'.	

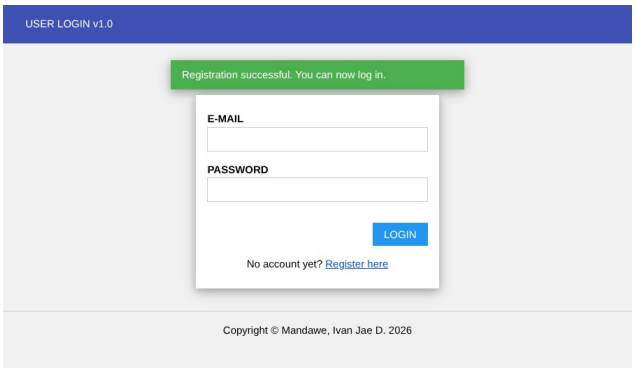
EX-03 Name made only of spaces

RESULT: FAIL

Procedure
Register with Name ' ' (three spaces), valid e-mail and password.

Expected result
Registration refused (a name is required).

Actual result
Rows saved with a blank name: 1.



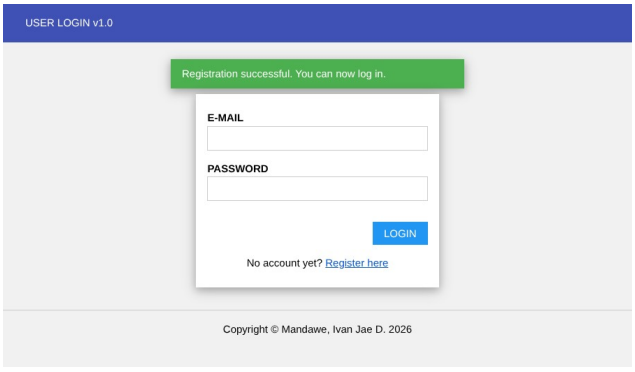
EX-04 Very short password

RESULT: FAIL

Procedure
Register with the one-character password 'a' (and confirmation 'a').

Expected result
Registration refused with a message about minimum password length.

Actual result
Message: 'Registration successful. You can now log in.'; rows saved with a 1-character password: 1.



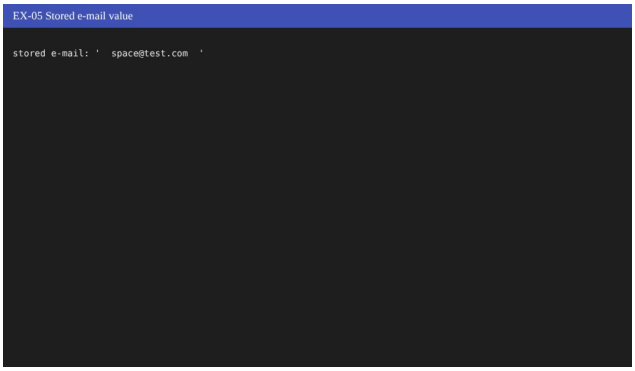
EX-05 E-mail typed with extra spaces

RESULT: FAIL

Procedure
Send a registration with e-mail ' space@test.com ' (spaces before and after).

Expected result
Spaces are removed before saving (stored e-mail is 'space@test.com').

Actual result
Stored e-mail value: ' space@test.com '.



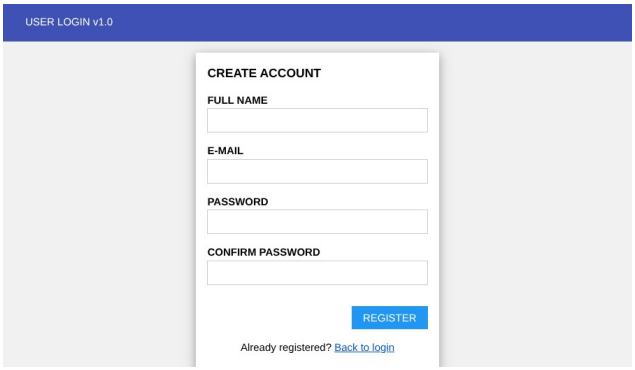
EX-06 Logged-in user opens the login/register pages

RESULT: FAIL

Procedure
Log in, then type / (and /register) in the address bar.

Expected result
User is sent straight to /main instead of seeing the login form again.

Actual result
Opening '/' ended at http://127.0.0.1:5057/; opening '/register' ended at http://127.0.0.1:5057/register.



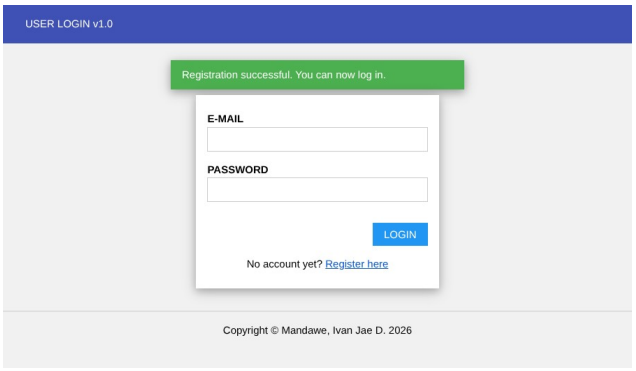
EX-07 Extremely long name (200 characters)

RESULT: FAIL

Procedure
Register with a 200-character name.

Expected result
Refused with a message (database column allows 50 characters).

Actual result
Message: 'Registration successful. You can now log in.'; rows saved: 1; longest stored name length: 200.



EX-08 Open /userlogin directly in the address bar

RESULT: PASS

Procedure
Type /userlogin (a POST-only route) in the address bar.

Expected result
Server handles it without crashing (405 Method Not Allowed is acceptable).

Actual result
HTTP 405 (no server crash).



EX-09 Open a page that does not exist

RESULT: PASS

Procedure
Open /abc123.

Expected result
HTTP 404 page, application keeps running.

Actual result
HTTP 404.



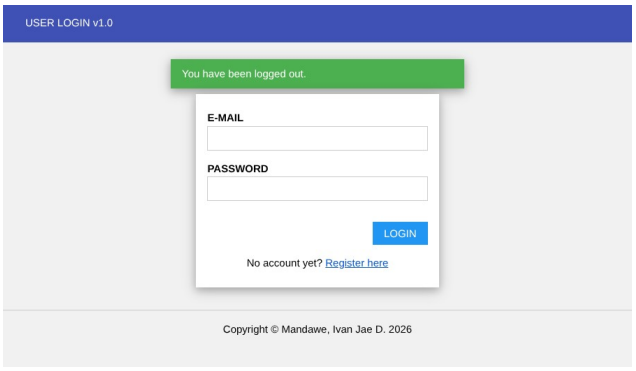
EX-10 Open /logout without being logged in

RESULT: PASS

Procedure
In a new session open /logout.

Expected result
No error; user lands on the login page.

Actual result
HTTP 200; login form visible: True.



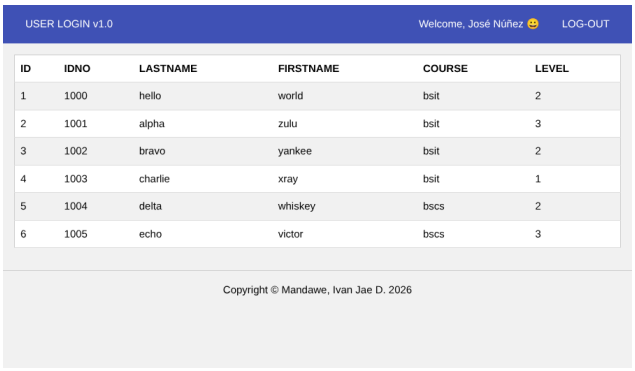
EX-11 Name with accents and emoji

RESULT: PASS

Procedure
Register the name 'José Núñez 😀', log in and look at the welcome text.

Expected result
Name is saved and displayed correctly.

Actual result
Header text: 'Welcome, José Núñez 😀'.



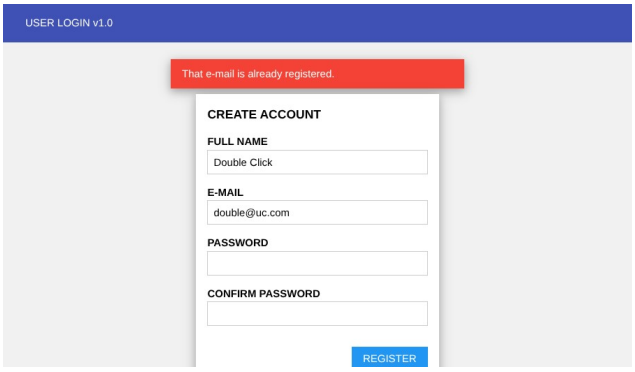
EX-12 Double submit of the register form

RESULT: PASS

Procedure
Send the same registration twice at the same moment.

Expected result
Only one account is created; no server error.

Actual result
Accounts created: 1; HTTP 200 and 200.



5. Defect log

Severity is the tester's own assessment: High = could let bad data or weak accounts into the system, Medium = wrong behaviour that users would notice, Low = minor or cosmetic.

ID	Found by	Description	Severity	Fix in v1.1	Retest
DEF-01	EX-01, EX-02	E-mail addresses were case-sensitive. Case@Test.com and case@test.com could both be registered, and a user who registered with capital letters could not log in by typing the e-mail in lower case.	Medium	E-mail is trimmed and converted to lower case before saving and before login (clean_email function).	PASS
DEF-02	EX-03	A name made only of spaces was accepted.	Low	Name is trimmed first, so a blank name is caught by the required-field check.	PASS
DEF-03	EX-04	A one-character password ("a") was accepted. There was no minimum length.	High	Passwords shorter than 8 characters are refused with a clear message.	PASS
DEF-04	EX-05	Spaces before or after the e-mail were saved as part of the address.	Low	Fixed by the same clean_email function as DEF-01.	PASS
DEF-05	EX-06	A logged-in user who opened / or /register saw the login or register form again.	Low	Both routes now redirect a logged-in user to /main.	PASS
DEF-06	EX-07	A 200-character name was saved although the database column is meant for 50 characters (SQLite does not enforce the limit).	Low	Name and e-mail longer than 50 characters are refused with a message.	PASS

6. Round 2: sanity testing (v1.1)

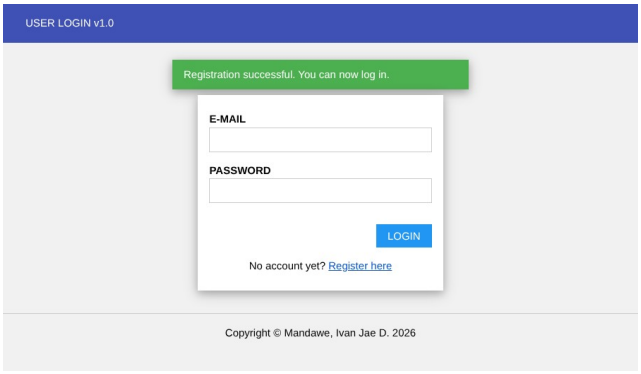
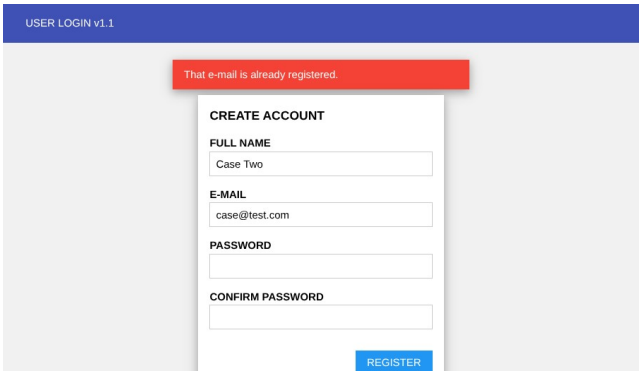
Right after the fixes, six quick cases were run on v1.1 to check that the build was sound before the longer retest: the start-up check, the critical login path, registering, logging in with a new account, logging out, and one of the fixed cases.

ID	Test case	Result	Actual result
SM-01	Application starts and home page loads	PASS	HTTP 200; title 'USER LOGIN'; header shows 'USER LOGIN v1.1'; login form visible: True.
SM-04	Critical path: seeded user can log in	PASS	Landed on http://127.0.0.1:5057/main; students table visible: True.
FT-01	Register with valid details	PASS	Now on http://127.0.0.1:5057/; message: 'Registration successful. You can now log in.'; rows in users for juan@uc.com: 1.
FT-02	Login with a newly registered account	PASS	URL http://127.0.0.1:5057/main; header text: 'Welcome, Ana Reyes'.
FT-08	Logout	PASS	After logout URL http://127.0.0.1:5057/; message 'You have been logged out.'; /main now redirects to login: True.
EX-02	Log in using different letter case than at registration	PASS	URL after login: http://127.0.0.1:5057/main; message: ''.

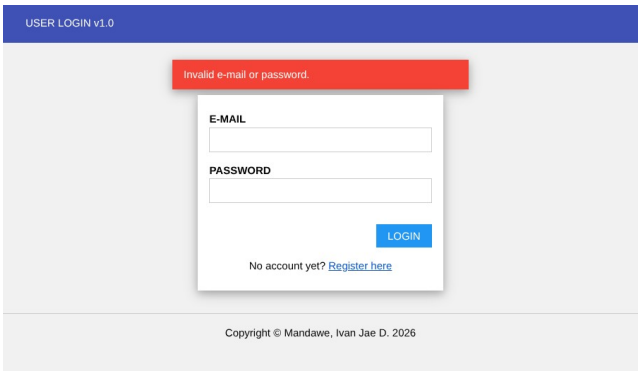
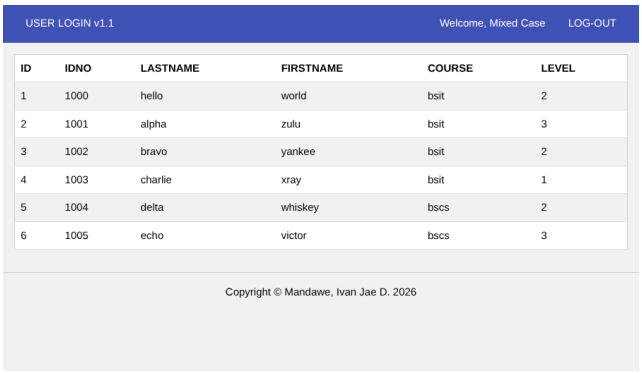
7. Round 2: confirmation testing (re-testing)

The seven test cases that failed in Round 1 were run again, step for step, on v1.1. The screenshots show the same test before the fix (left) and after the fix (right).

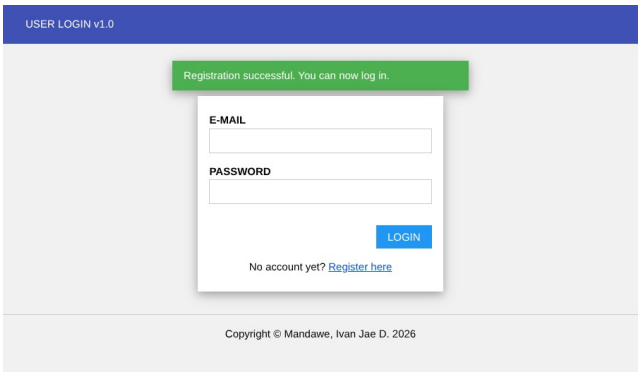
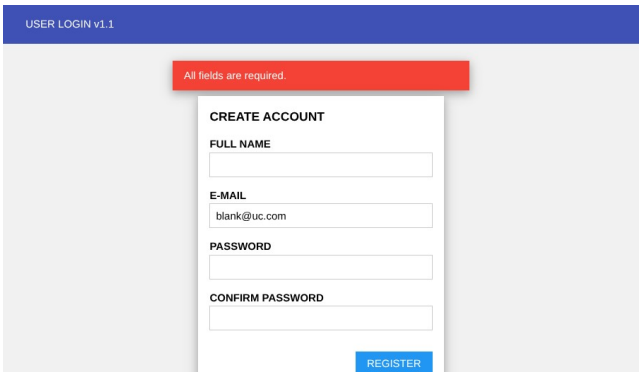
EX-01 Same e-mail typed with different letter case (DEF-01)

BEFORE (v1.0): FAIL	AFTER (v1.1): PASS
	
Expected: Second registration refused (e-mail addresses are not case-sensitive). Actual: Message: 'Registration successful. You can now log in.'; accounts now stored for this address: 2.	Actual: Message: 'That e-mail is already registered.'; accounts now stored for this address: 1.

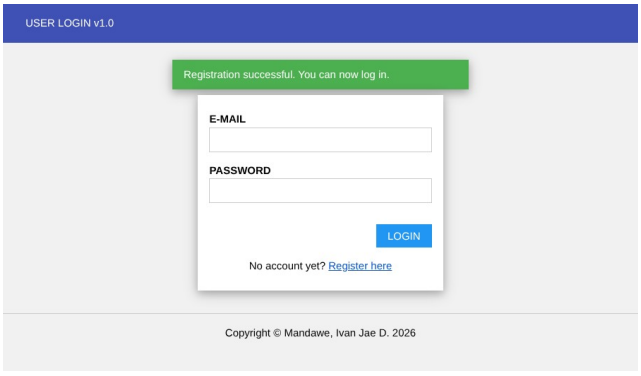
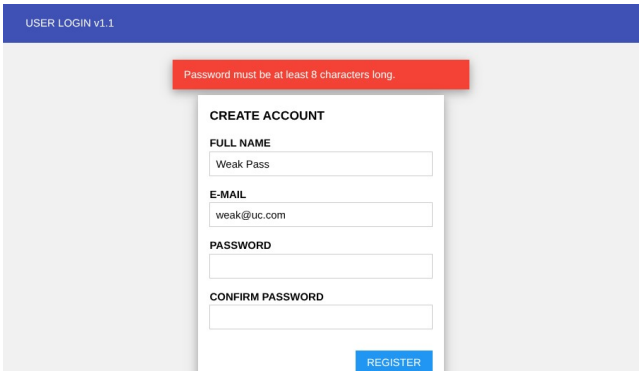
EX-02 Log in using different letter case than at registration (DEF-01)

BEFORE (v1.0): FAIL	AFTER (v1.1): PASS
	
Expected: Login succeeds. Actual: URL after login: http://127.0.0.1:5057/; message: 'Invalid e-mail or password.'.	Actual: URL after login: http://127.0.0.1:5057/main; message: ''.

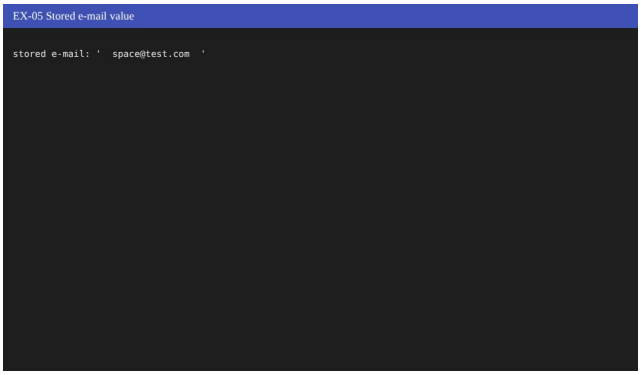
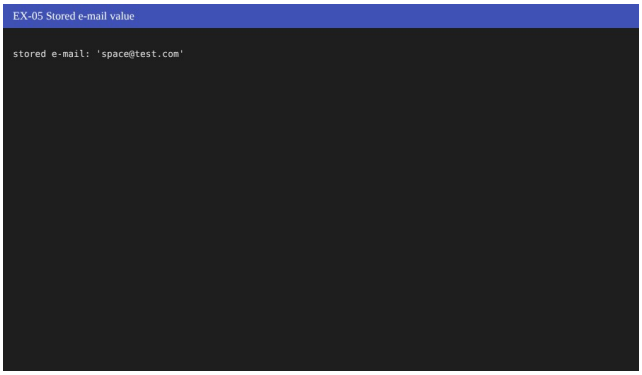
EX-03 Name made only of spaces (DEF-02)

BEFORE (v1.0): FAIL	AFTER (v1.1): PASS
	
Expected: Registration refused (a name is required). Actual: Rows saved with a blank name: 1.	Actual: Rows saved with a blank name: 0.

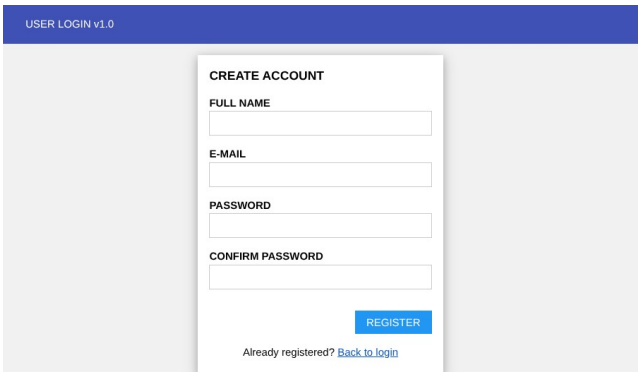
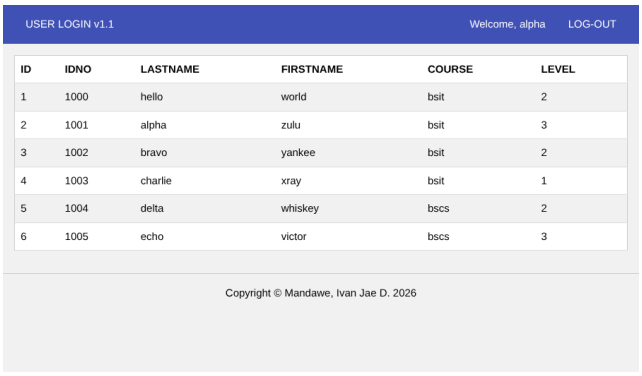
EX-04 Very short password (DEF-03)

BEFORE (v1.0): FAIL	AFTER (v1.1): PASS
	
Expected: Registration refused with a message about minimum password length. Actual: Message: 'Registration successful. You can now log in.'; rows saved with a 1-character password: 1.	Actual: Message: 'Password must be at least 8 characters long.'; rows saved with a 1-character password: 0.

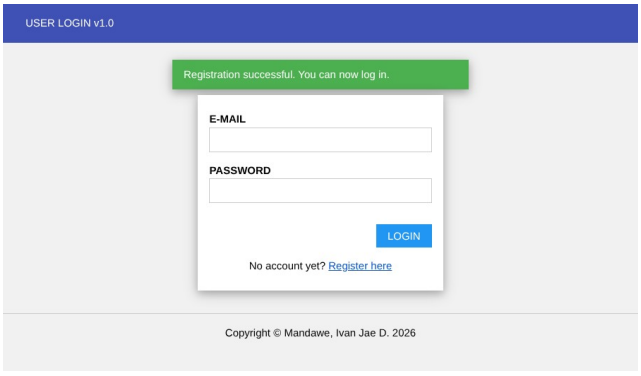
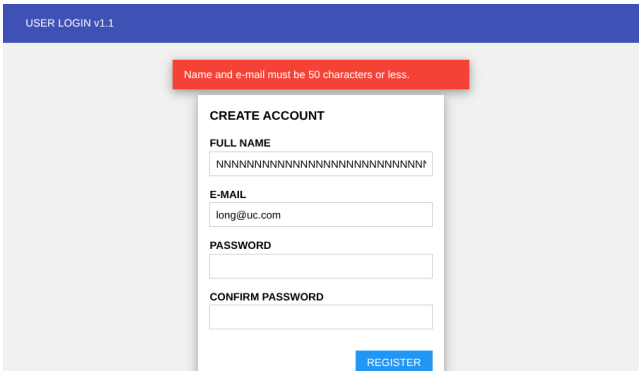
EX-05 E-mail typed with extra spaces (DEF-04)

BEFORE (v1.0): FAIL	AFTER (v1.1): PASS
	
Expected: Spaces are removed before saving (stored e-mail is 'space@test.com'). Actual: Stored e-mail value: ' space@test.com '.	Actual: Stored e-mail value: 'space@test.com'.

EX-06 Logged-in user opens the login/register pages (DEF-05)

BEFORE (v1.0): FAIL	AFTER (v1.1): PASS
	
Expected: User is sent straight to /main instead of seeing the login form again. Actual: Opening '/' ended at http://127.0.0.1:5057/; opening '/register' ended at http://127.0.0.1:5057/register.	Actual: Opening '/' ended at http://127.0.0.1:5057/main; opening '/register' ended at http://127.0.0.1:5057/main.

EX-07 Extremely long name (200 characters) (DEF-06)

BEFORE (v1.0): FAIL	AFTER (v1.1): PASS
	
Expected: Refused with a message (database column allows 50 characters). Actual: Message: 'Registration successful. You can now log in.'; rows saved: 1; longest stored name length: 200.	Actual: Message: 'Name and e-mail must be 50 characters or less.'; rows saved: 0; longest stored name length: None.

8. Round 2: regression testing (v1.1)

The complete suite of 37 test cases was run again on v1.1 to make sure the fixes did not break any feature that was working before. Screenshots for the regression run are saved in the SemiFinal folder (tests/results/regression-v1.1).

ID	Type	Test case	v1.0 (Round 1)	v1.1 (regression)
SM-01	Smoke	Application starts and home page loads	PASS	PASS
SM-02	Smoke	Register page is reachable from the login page	PASS	PASS
SM-03	Smoke	Stylesheet loads	PASS	PASS
SM-04	Smoke	Critical path: seeded user can log in	PASS	PASS
SM-05	Smoke	Footer shows the correct copyright name	PASS	PASS
FT-01	Functional	Register with valid details	PASS	PASS
FT-02	Functional	Login with a newly registered account	PASS	PASS
FT-03	Functional	Login with a wrong password	PASS	PASS
FT-04	Functional	Login with an unregistered e-mail	PASS	PASS
FT-05	Functional	Register with an e-mail that already exists	PASS	PASS
FT-06	Functional	Register with mismatched passwords	PASS	PASS
FT-07	Functional	Required field validation (server side)	PASS	PASS
FT-08	Functional	Logout	PASS	PASS
FT-09	Functional	Protected page without logging in	PASS	PASS
FT-10	Functional	Main page lists all students	PASS	PASS
FT-11	Functional	Session persists after page refresh	PASS	PASS
FT-12	Functional	'Back to login' link on the register page	PASS	PASS
NF-01	Non-functional	Performance: page response time	PASS	PASS
NF-02	Non-functional	Security: passwords are stored hashed	PASS	PASS
NF-03	Non-functional	Security: SQL injection on login	PASS	PASS
NF-04	Non-functional	Security: script injection (XSS) in the name field	PASS	PASS
NF-05	Non-functional	Security: Back button after logout	PASS	PASS
NF-06	Non-functional	Security: session cookie flags	PASS	PASS
NF-07	Non-functional	Usability: mobile layout (375 px wide)	PASS	PASS
NF-08	Non-functional	Reliability: 25 consecutive logins	PASS	PASS
EX-01	Exploratory	Same e-mail typed with different letter case	FAIL	PASS
EX-02	Exploratory	Log in using different letter case than at registration	FAIL	PASS
EX-03	Exploratory	Name made only of spaces	FAIL	PASS
EX-04	Exploratory	Very short password	FAIL	PASS
EX-05	Exploratory	E-mail typed with extra spaces	FAIL	PASS
EX-06	Exploratory	Logged-in user opens the login/register pages	FAIL	PASS
EX-07	Exploratory	Extremely long name (200 characters)	FAIL	PASS
EX-08	Exploratory	Open /userlogin directly in the address bar	PASS	PASS
EX-09	Exploratory	Open a page that does not exist	PASS	PASS
EX-10	Exploratory	Open /logout without being logged in	PASS	PASS
EX-11	Exploratory	Name with accents and emoji	PASS	PASS
EX-12	Exploratory	Double submit of the register form	PASS	PASS

Result: 37 of 37 test cases passed on v1.1. No regressions were found.

9. Exploratory testing notes

Exploratory testing was done with three short charters. The session notes below summarise what was tried and what was learned.

Charter	Cases	What was learned
Explore how the registration form handles unusual input	EX-01, EX-03, EX-04, EX-05, EX-07, EX-11, EX-12	Five problems found (letter case, blank name, short password, extra spaces, over-long name). Accents/emoji (EX-11) and a double submit (EX-12) were handled correctly. The UNIQUE constraint on the e-mail column prevented duplicate rows on double submit.
Explore login behaviour with different e-mail forms	EX-02	Logging in with a different letter case than the one used at registration failed. This led to DEF-01.
Explore navigation and error handling	EX-06, EX-08, EX-09, EX-10	Wrong URLs return proper 404/405 pages and /logout without a session is harmless. A logged-in user could still see the login and register forms (DEF-05).

10. Conclusion

The user login system works as required. Registration, login, logout and the protected page behave correctly, passwords are stored as salted hashes, SQL injection and script injection attempts were blocked, the session cookie is HttpOnly, the Back button does not reveal protected content after logout, and pages respond in a few milliseconds on localhost.

The first build had six defects in input handling and navigation, found mainly through exploratory testing. They were fixed in v1.1 and confirmed by re-testing, and the regression run shows no side effects. The exploratory results also show why scripted tests alone are not enough: the 30 scripted cases that were written first all passed on v1.0, while the unusual-input sessions found the defects.

10.1 Limits of this testing and suggested improvements

- Only Chromium was used. Other browsers (Firefox, Safari, mobile browsers) were not tested.
- The tests ran against the Flask development server on localhost, so the response-time results are not a measure of real-world performance.
- The forms have no CSRF protection and the login has no limit on repeated wrong passwords. These were not tested and should be added before real use.
- The session secret key has a default value in app.py. For real use it should be set through the SECRET_KEY environment variable and the cookie should be marked Secure when HTTPS is used.
- There is no password reset or e-mail verification; both are outside the scope of this activity.

Appendix A. How to run the application

Open a terminal in the SemiFinal/userlogin folder and run:

```
python -m venv venv
venv\Scripts\activate        (Windows)
python -m pip install flask
python app.py
```

Then open http://127.0.0.1:5000 in a browser. The database (db/school.db) and the sample data are created automatically the first time the app runs. Sample login: alpha@uc.com / hello123, or use the Register page to create a new account.

Appendix B. Contents of the SemiFinal folder

Path	Description
SemiFinal/Test_Report.pdf	This report (PDF).
SemiFinal/Test_Report.docx	This report (Word version).
SemiFinal/userlogin/app.py	Flask application: routes for login, register, main page and logout.
SemiFinal/userlogin/db/dbhelper.py	SQLite helper functions and automatic creation of the tables and sample data.
SemiFinal/userlogin/templates/	base.html, index.html (login), register.html, main.html.
SemiFinal/userlogin/static/css/w3.css	Stylesheet (W3.CSS-compatible subset).
SemiFinal/tests/run_tests.py	Automated test script (Playwright) used to execute all test cases.
SemiFinal/tests/results/	Screenshots and results.json for each run: round1-v1.0, sanity-v1.1, confirm-v1.1, regression-v1.1.